

Claude Code 控制流先落地 RUNTIME DISCIPLINE	Codex 制度层先设防 POLICY AND LOCAL RULES
秩序住在运行时	规矩住在系统外沿

Claude Code 和 Codex 的 Harness 设计哲学

殊途同归，还是各表一枝

做系统的人，
不过是把必然会来的摧残，
提前写进控制流，
省得它反过来支配你。

从会用 Agent, 到做出 Agent PoC

<https://agentway.dev>

目录

导读	1
阅读地图	2
先看第一本书：Claude Code 给出的九个结构判断	2
再看这本比较书：同一个问题，Claude Code 和 Codex 从哪里起笔	3
合起来以后，真正清楚的是什么	4
推荐阅读顺序	5
一句话总结	5
序言两套 Harness	6
第 1 章为什么要把 Claude Code 和 Codex 放在一起看	8
1.1 因为它们比较的是对模型的不信任	8
1.2 Claude Code 代表一种运行时优先的驯化路线	8
1.3 Codex 代表一种显式控制面优先的驯化路线	9
1.4 两者都在回答同一个问题，但起笔位置不同	10
1.5 为什么这种差异值得团队认真看	10
1.6 本文的基本判断	11
第 2 章两种控制面	12
2.1 控制面这件事，首先不是文风问题	13
2.2 Claude Code 的控制面是动态装配线	13
2.3 Codex 的控制面是带编号的公文系统	13
2.4 AGENTS.md 与 CLAUDE.md：同样是本地规则，气质却不同	15
2.5 两种控制面的代价	15
2.6 这章的比较结论	15

第 3 章心跳放在哪	16
3.1 代理系统的核心是连续性	17
3.2 Claude Code: 把连续性压进主循环	17
3.3 Codex: 把连续性拆成线程、rollout 与状态桥	17
3.4 差别在于状态安放的位置	19
3.5 对恢复与可审计性的影响	20
3.6 对产品和团队接口的影响	21
3.7 本章结论	21
第 4 章工具、沙箱与策略语言	22
4.1 真正危险的是开始执行	22
4.2 Claude Code: 重点在运行时编排和危险动作约束	23
4.3 Codex: 重点在工具 schema、审批参数和策略引擎	23
4.4 运行时审批对照策略语言	23
4.5 沙箱与审批: 产品定义问题	25
4.6 MCP、扩展工具与边界外移	25
4.7 本章结论	25
第 5 章技能、Hook 与本地规则	27
5.1 真正能落地的 agent, 一定会地方化	27
5.2 Claude Code: 把局部制度做成现场记忆	28
5.3 Codex: 把局部制度做成结构化注入和事件系统	28
5.4 Claude Code 偏经验收编, Codex 偏制度挂载	28
5.5 对组织可复制性的影响	29
5.6 本章结论	29

第 6 章委派、验证与持久状态	31
6.1 多代理的真正问题是责任	31
6.2 Claude Code: 多代理服务于运行时职责分区	31
6.3 Codex: 多代理服务于显式工具化协作	31
6.4 持久状态让验证不只是礼仪	32
6.5 对恢复与收尾的不同态度	32
6.6 本章结论	33
第 7 章殊途同归	34
7.1 先说“同归”的部分	34
7.2 再说“各表一枝”的部分	34
7.3 如果非要给它们起一个更难听但更准确的名字	35
7.4 对后来者的启发	35
7.5 最终判断	36
第 8 章如果你要自己做	37
8.1 比较的最终用途是少走弯路	37
8.2 三种常见团队，三种起手方向	37
8.3 什么该学 Claude Code，什么该学 Codex	39
8.4 一个危险误区：把”显式”与”灵活”误认为天然对立	40
8.5 给后来者的一组顺序建议	40
8.6 本章结论	40
附录 A 源码地图	42
A.1 Claude Code 侧主要依据	42
A.2 Codex 侧主要依据	43

A.3 各章对照	44
附录 B 检查清单	46
B.1 控制面清单 (invariants)	46
B.2 连续性清单 (invariants)	46
B.3 工具与审批清单 (invariants)	47
B.4 本地治理清单 (invariants)	47
B.5 多代理与验证清单 (invariants)	47
B.6 你更像哪一类系统	47
B.7 最后六问	48
B.8 阈值与顺序速查 (thresholds & orderings)	48

导读

“人活在世上，就是为了忍受摧残。”

做 harness 也是。区别只在于，有人把摧残写进控制流，有人把摧残写进制度层。

这不是一份功能表，也不是一篇产品评测。它要比较的是两套系统如何承认模型不可靠，并把这种不可靠驯化成可持续工作的工程秩序，而不只是看“谁支持更多工具”。

本书有三个判断前提：

- Claude Code 和 Codex 比较的重点不在模型，而在 harness
- harness 是一种权力分配方式，而不是若干功能的拼盘
- 工程系统的差别，常常不在名词，而在秩序住在哪一层

如果你第一次进入这套比较，建议这样读：

1. 先读序言，确认比较对象不是模型，而是 harness 的秩序设计。
2. 再读第 1 到第 6 章，顺着控制面、连续性、工具治理、本地制度和多代理验证五条比较轴往下走。
3. 然后读第 7 章殊途同归，还是各表一枝，把前面的比较收束成总判断。
4. 最后读第 8 章如果你要自己做：该向谁学，先学什么，把比较转成可执行的路径选择。

如果只想先看总判断，可以直接跳到第 7 章。

阅读地图：如何理解第一本书与这本比较书

如果把这两个目录都看成在写 AI coding agent，那么最容易误读的地方，是把它们当成两份重复文档。它们并不重复，分工其实很清楚。

第一本书更像单体解剖。它以 Claude Code 为样本，讨论一套可控 agent 为什么必须具备控制面、query loop、工具权限、上下文治理、恢复路径、多代理验证和团队制度这些器官。它要回答的问题是：一套能在真实工程环境里持续工作的 harness，内部骨架应该是什么样。

这本比较书更像比较解剖。它把 Claude Code 和 Codex 放在一起，比较两者如何承认模型不可靠，以及如何把秩序安放在不同层级。它要回答的问题是：同样是做 harness，哪些设计更接近共识，哪些体现了不同工程路线的取舍。

现在还可以再多补一层用途：当你把前面这些判断带去看第三方 harness 时，也更容易识别一种常见误区。很多系统表面上也有 memory、skills、compact 和多代理，但上下文治理的主轴仍然是先把大量文本塞进 prompt，超了再截断和补救。这种路线看上去像“信息更全”，实际往往更费 token，也更容易把工作语义冲淡。

换句话说，可以把两者理解成同一研究计划的前后两步：

- 第一步，在第一本书里先抽出 Harness Engineering 的一般原则。
- 第二步，在这本比较书里再看这些原则如何在两套系统里分别落地。

先看第一本书：Claude Code 给出的九个结构判断

第一本书的主线可压成九条：harness 先约束模型别把工程环境弄坏；prompt 是控制平面的一部分；query loop 才是 agent 的心跳；工具是受审批、调度、中断语义约束的执行接口；上下文越多不等于越好，memory / CLAUDE.md / compact 本质是预算治理；错误属于主路径，恢复要第一等设计；多代理的价值在职责分区与独立验证；团队

落地必须把规则沉淀成可复用制度；最终这些抽成一套稳定的 Harness Engineering 原则清单。

只读第一本书会得到一个强印象：Claude Code 的系统气质偏运行时治理——它先关心会话如何连续运行、工具如何别闯祸、恢复如何别把系统拖进死循环、验证如何别沦为形式。

再看这本比较书：同一个问题，Claude Code 和 Codex 从哪里起笔

这本比较书在这个基础上，把比较轴明确拆成了几层。

控制面

Claude Code 更像动态 prompt 装配线。它把很多秩序压进运行前后的 prompt 拼装、会话状态和上下文治理里。

Codex 更像显式控制层。它把 instruction fragments、审批策略、工具 schema、thread、rollout、hook 等结构尽量模块化、类型化、可组合化。

连续性

Claude Code 把连续性压进主循环，强调 query loop 的心跳纪律。

Codex 把连续性拆进 thread、rollout 和 state bridge，强调状态如何被结构化持有和恢复。

工具与权限

Claude Code 偏运行时约束，重点在调用时如何审批、如何中断、如何避免危险动作直接落地。

Codex 偏策略语言和工具契约，重点在 schema、approval policy、sandbox 和 exec policy 这些显式器官。

本地治理

Claude Code 倾向把地方经验收编成现场记忆，例如 `CLAUDE.md`、`memory`、`skills` 和工作流约束。

Codex 倾向把地方制度挂到结构化注入和事件系统上，例如 `instructions`、`skills`、`hooks` 和明确的工具边界。

多代理与验证

Claude Code 强调多代理是运行时职责分区，验证必须独立于实现阶段。

Codex 则更强调通过显式委派、持久状态和工具化协作，把验证从“礼仪动作”变成可跟踪的系统能力。

合起来以后，真正清楚的是什么

把第一本书和这本比较书放在一起看，能得到三个更完整的结论。

第一，比较的重点主要落在 harness

这两套文档反复指向同一个事实：AI coding system 的核心挑战，首先是如何让模型在终端、文件系统、权限和团队制度中不失控；模型能力的提升要放在这个前提里理解。

第二，Claude Code 和 Codex 的差别，主要是秩序安放的位置不同

Claude Code 更像从运行时事故经验里塑出来的系统，优先解决连续性、恢复和现场治理。

Codex 更像从显式结构设计里塑出来的系统，优先解决控制层命名、策略表达、边界清晰和可组合性。

第三，后来者不该照抄产品，而该识别自己的主要不确定性

如果你的问题是长会话容易失控、恢复路径很脆、验证总被跳过，那么更该先学第一本书强调的运行时纪律。

如果你的问题是规则来源太散、权限边界不清、工具契约不稳定、团队很难复制同一套行为，那么更该先学这本比较书里总结出的 Codex 式显式控制层。

如果你看到某套系统主要靠堆叠 bootstrap 文本、身份设定、技能目录和工作区说明来维持连续性，那也不必急着被“上下文很多”这件事打动。很多时候，这更像一种尚未把上下文真正治理起来的过渡状态，离成熟的第三条路线还有一段距离。

推荐阅读顺序

如果你是第一次进入这套材料，推荐按下面的顺序读。

1. 先读序言，先确认这套比较关注的是秩序放在哪一层，不只是罗列功能表。
2. 再读第 1 章为什么要把 Claude Code 和 Codex 放在一起看，建立问题意识。
3. 然后顺着第 2 到第 6 章读完控制面、连续性、工具治理、本地制度和多代理验证这五条比较轴。
4. 如果只想快速看总结，直接读第 7 章殊途同归，还是各表一枝。
5. 如果你的目标是自己动手搭系统，最后读第 8 章如果你要自己做：该向谁学，先学什么。这一章现在也补了一张三路对照图，用来说明为什么有些 harness 明明“上下文很满”，却依然又贵又乱。

一句话总结

第一本书解释的是：为什么一个可控的 agent 必须采用这种结构。

这本比较书解释的是：当两套系统都认真做 harness 时，它们为什么会长得不一样。

序言两套 Harness, 不必假装是同一匹马的附件

比较两套 AI coding harness, 最容易犯的错误, 是拿一张功能对照表当作思想史。左边写“有技能”, 右边也写“有技能”; 左边写“有沙箱”, 右边也写“有沙箱”; 左边写“能开子代理”, 右边也写“能开子代理”。这样写的好处是省事, 坏处是几乎什么也没说。因为工具栏上的名词相同, 不代表系统的骨架相同。就像两个城市都修了桥, 不能说明它们是按同一条河设计的。

这份内容想比较的是骨头, 而不只是名字。

Claude Code 和 Codex 都属于那一类已经不满足于“让模型多说两句”的系统。它们都把模型放进了更麻烦的环境里: 终端、文件系统、权限、工具调用、团队规范、长会话、恢复路径。到了这一步, 真正决定系统成败的, 往往就不再是模型本身, 而在于它外面那套不怎么讨喜的约束装置。也就是 harness。

我在前一套内容里, 已经把 Claude Code 当作一个样本, 讨论过 Harness Engineering 的一般原则。那时重点是单体解剖。现在再看, 会觉得还缺一个动作: 把它和另一个同样认真、但长法不同的系统摆在一起。只有比较以后, 很多原来看似自然的设计, 才会暴露出它其实只是某一条工程路径的选择。

Codex 值得比较, 恰恰因为它不是 Claude Code 的复制版。从 `core/src/lib.rs` 一眼看过去, 就能看到一种很明确的系统意志: 把线程、rollout、state bridge、instructions、skills、hooks、sandboxing、exec policy、tools 这些东西拆成模块。目标是让控制层显式地变成可组合、可导入、可序列化、可策略化的器官, 而不只是藏在一团运行时直觉里。

Claude Code 则不同。它更接近在主循环的压力下被塑出来的系统。你去看 `src/query.ts`, 再看它周围包裹的 compact、tool orchestration、permission、中断与恢复, 很容易意识到, 它的很多精华都发生在“这轮怎么接下一轮”这个问题上。也就是说, 它的 harness 首先得连续运转, 然后才谈如何把规则拆得更漂亮。

两者都承认模型不可靠。这一点很重要。因为只要这一点成立, 许多浪漫的说法就都得后退。你不能把模型当作一个天然可信的执行体, 不能让它在无边界条件下操作 shell、

文件和网络，也不能指望它凭一时聪明维持长会话的秩序。一个系统要是愿意承认这一点，它就早晚会补齐 prompt 分层、状态持久化、权限控制、上下文治理、失败恢复、验证机制和本地制度这些器官。只是器官的位置，不一定一样。

这份内容还有一个边界，需要先说清楚。它不会提供 Claude Code 或 Codex 的源代码副本，也不会大段转录实现文本。原因并不玄妙，无非是版权和边界感。我们能做的，是在合理引用和工程分析范围内，基于文件结构、模块接口和可观察的设计意图，提炼出比较结论。换句话说，这里讨论的是系统怎样思考自己，而不是把受保护的实现挪到别处摆出来。

如果一定要先给出一句总判断，我现在比较愿意这样说：

Claude Code 与 Codex 的共同点，不在于它们都很会调用工具，而在于它们都不肯把模型当作一个可以放任自流的部件。

至于不同点，则更耐看一些。

Claude Code 更接近从运行时纪律出发。它担心会话怎么断，工具怎么串，compact 怎么救，用户怎么插话，子代理怎么收尸。它有一种做过很多脏活以后的现实感，像个知道现场会出乱子的人，因此把很多本该由“聪明”解决的问题，提前交给控制流和恢复流去解决。

Codex 更接近从结构化控制层出发。它把 instruction fragment、thread、approval policy、tool schema、hook event、exec policy 这些东西尽量明说出来，不羞于把系统做成一套带强命名、强边界和强配置感的装置。它不相信默契，宁可多定义几个类型，也不愿靠一团含混的约定去维持系统秩序。

两种路数都合理。只是合理的地方，不一样。

所以这不是一篇谁输谁赢的文章。工程上真正有价值的比较，通常是识别路径依赖，而不是排座次。你用什么结构去承认模型的不可靠，就会形成什么样的 harness。你把控制权放在哪一层，系统以后就会围着哪一层长肉。很多团队做 agent 总爱问“有没有最佳实践”，这问题问得太像买厨具。更好的问题应该是：你的主要不确定性在哪，你准备把秩序安放在哪。

下面七章，就从这个问题开始。

@wquguru

2026.04.01

Claude Code 源码泄漏的愚人节

btw. 您可以在 harness-books.agentway.dev/book2-comparing 访问在线版，获取更好的阅读体验

第 1 章为什么要把 Claude Code 和 Codex 放在一起看

1.1 因为它们比较的是对模型的不信任

如果把 Claude Code 和 Codex 当作两个“会写代码的助手”，那比较就会很无聊。无非是谁支持更多工具、谁的终端体验更顺手、谁的配置项更细。这些东西当然有用，但它们不解释为什么系统会呈现出今天这种形态。

真正值得比较的地方，是它们都承认了一件对产品宣传不太友好的事实：模型不能直接拿来当执行体。它会误判，会忘记上下文，会把语气里的自信和结论里的正确性混为一谈。一旦这种东西接上 shell、文件系统和多轮状态，问题就不再是“答得对不对”，而是“它做过的事以后谁来收拾”。

Claude Code 在这一点上表现得很像一个做过事故复盘的系统。你看 `query.ts`、`toolOrchestration.ts`、`compact.ts` 以及 Bash 相关 prompt，就会发现它对失败、膨胀、中断和误操作的想象力很丰富。一个系统如果老是预设好几种失败路径，通常是因为它见过太多乐观主义留下的烂摊子，而不只是因为悲观。

Codex 则是另一种诚实。它把“不信任模型”落实为显式模块边界。`core/src/lib.rs` 里把 `instructions`、`skills`、`sandboxing`、`exec_policy`、`rollout`、`state_db_bridge`、`thread_manager` 等器官明明白白列出来，等于是在说：秩序不能靠模型自己领会，得靠系统把职责拆出来。

所以比较这两者，是为了看两种不同的驯化路线，而不是为了看谁更会调用命令。

1.2 Claude Code 代表一种运行时优先的驯化路线

Claude Code 的很多精华，发生在主循环附近。这不是偶然。因为它的问题意识本来就强烈地围绕“会话如何连续”展开。系统不只是一要回答，还要在回答之后继续存在；不只是一要执行，还要在执行之后保留秩序。这种压力会让架构自然向运行时编排倾斜。

因此它的关键词是：

- query loop
- compact
- tool orchestration
- interrupt
- permission ask/allow/deny
- forked agent lifecycle

这些词看上去都不像品牌文案，更接近值班表。可一个系统到了真的要干活的时候，值班表往往比愿景更重要。

Claude Code 的长处也因此很鲜明。它对“连续工作中的脏活”特别敏感：消息膨胀、输出截断、工具结果回灌、上下文裁剪、子代理状态隔离、验证阶段独立、hook 生命周期收口。这些东西不是光靠一套漂亮接口定义就能解决的，必须经过运行时一轮轮摔打。

1.3 Codex 代表一种显式控制面优先的驯化路线

Codex 的气质不太一样。它当然也有运行时，但更引人注目的，是它把控制层尽量做成显式结构。

在 `instructions/src/fragment.rs` 和 `user_instructions.rs` 里，用户指令和 skill 指令会以带有清楚 marker 的 contextual fragment 进入 prompt，而不是随便拼接。AGENTS.md 的内容会被包装成明确边界的用户消息，skill 也会以 `<skill>` 片段注入。这说明 Codex 对控制面有一种近乎文书工作的偏爱：它并不拒绝灵活，但不愿让灵活失去边界。

同样，`tools/src/lib.rs` 和 `local_tool.rs` 也很说明问题。工具是一组 schema 化的构造结果，而不只是一堆运行时对象。`exec_command`、`shell`、`write_stdin`、`request_permissions`、`spawn_agent`、`wait_agent` 等工具，在系统里首先是接口定义，其次才是执行动作。

再看 `execpolicy/src/lib.rs`，审批和执行限制甚至被做成单独 crate，包含 Policy、Rule、Evaluation、Decision、parser 等概念。这种做法其实很少见。很多系统也有权限策略，但往往散落在运行时各处；Codex 则把它往“可解析的政策语言”那一边推了一步。

1.4 两者都在回答同一个问题，但起笔位置不同

从更高处看，这两套系统都在回答同一个问题：

如何让一个会说话、会调工具、但本质不可靠的模型，在工程环境里做出可接受的事？

Claude Code 的起笔位置更接近：

“模型已经在循环里了。我们怎样保证它这一轮不会把下一轮弄坏？”

Codex 的起笔位置更接近：

“模型会接触很多控制信息。我们怎样把这些信息做成显式、可组合、可策略化的结构？”

前者强调流转，后者强调分层。前者把秩序安在 runtime heartbeat 上，后者把秩序安在 typed substrate 上。两者都不轻信模型，但它们把“不信任”安放在不同地方。

1.5 为什么这种差异值得团队认真看

因为一个团队以后会形成什么样的工作方式，常常取决于它最先把秩序放在哪。

如果你先强调运行时连续性，团队自然会更关注：

- 错误恢复
- 中断响应
- 状态污染
- 工具编排
- 长会话可靠性

如果你先强调显式控制面，团队自然会更关注：

- 指令边界
- 配置层级
- tool schema
- policy 语言
- thread state 与持久化

两种方向都没有错。错的是混着用，最后既没有运行时纪律，也没有清楚的控制层，只剩一堆可以演示、但不好维护的聪明把戏。

1.6 本文的基本判断

这一章先把全文立场摆明：

Claude Code 和 Codex 代表的是两种 harness 设计哲学，而不只是两个“谁功能更全”的产品。

它们殊途同归，是因为都知道模型不可信，真正可信的只能是约束结构。

它们各表一枝，是因为一个更接近从现场经验里塑出来的运行时系统，另一个更接近由类型、策略和线程模型塑出来的控制系统。

下面的章节，就沿着这条判断往下拆。

第 2 章两种控制面：Prompt 拼装与 Instruction Fragment

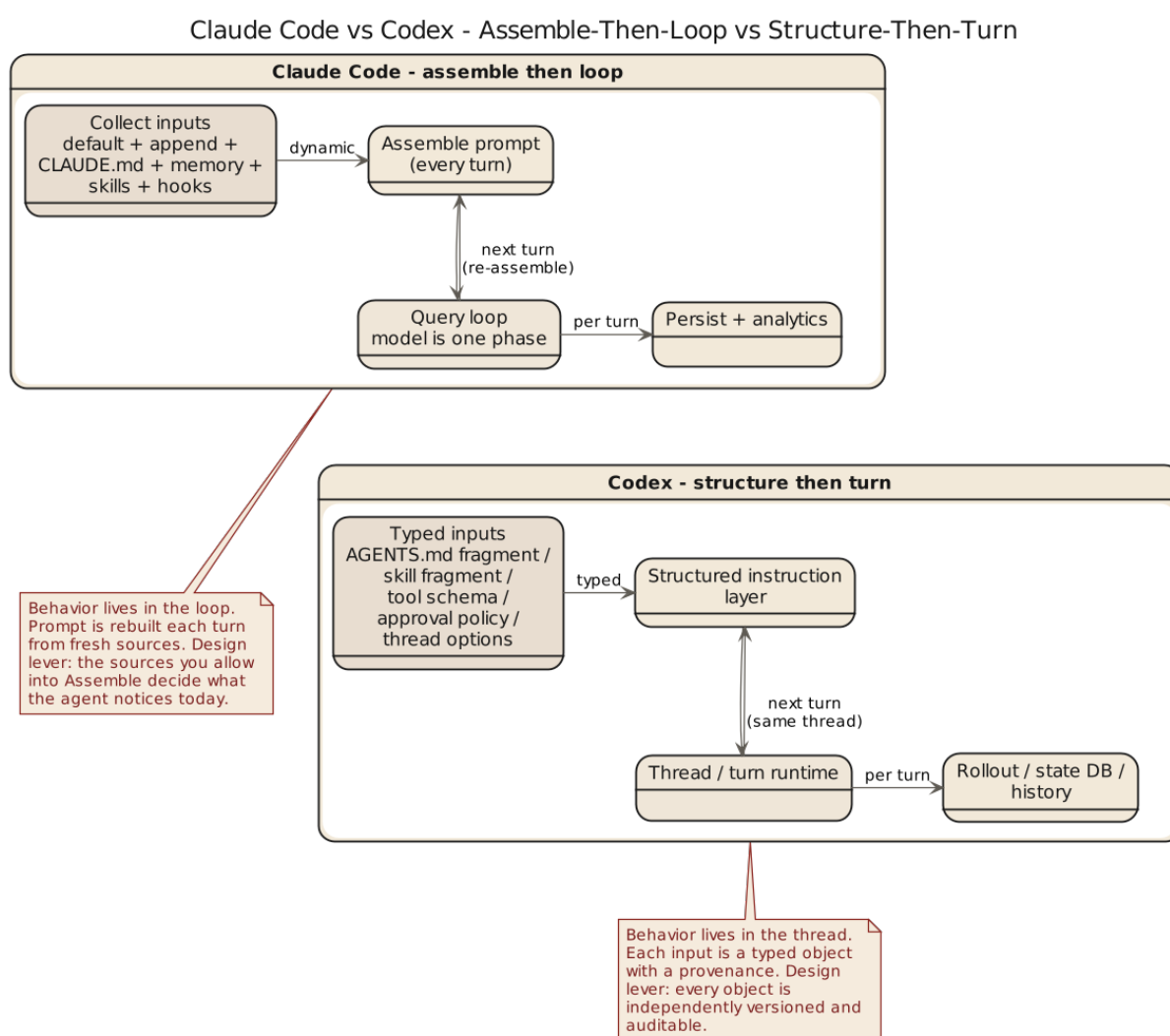


图 1: 控制面对比图

2.1 控制面这件事，首先不是文风问题

谈 prompt 最容易落到文风上,仿佛系统控制的核心是把一段文字写得更像老工程师——这有点像把警察制度理解成说话语气。Claude Code 和 Codex 都不这么看,它们都把面向模型的指令当作行为控制的一部分,只是实现方式不同。Claude Code 采用层层拼装: `constants/prompts.ts`、`utils/systemPrompt.ts`、`claudemd.ts`、`memory` 与 `output style` 按运行时条件注入到 `system prompt` 里,重点不是文案而是多个来源如何排优先级、避免互相打架。Codex 则是结构化片段: `instructions/src/fragment.rs` 定义 `AGENTS.md` 和 `skill` 的 `fragment` 标记, `user_instructions.rs` 把用户指令序列化成带目录和边界标记的消息——`instruction` 不是随意串接的自由文本,而是“有开始、有结束、有来源类型”的上下文单元。

2.2 Claude Code 的控制面是动态装配线

Claude Code 的 `system prompt` 有一个朴素前提:控制面会随着任务、`memory`、工具能力和团队注入而变化,不是固定文本。因此它是装配线——默认 `prompt` 做底板, `append prompt` 加要求, `agent prompt` 补角色, `CLAUDE.md` 和 `memory` 带现场条件。好处是同一主循环适配多场景;代价是必须在乎装配顺序,否则层层叠加容易互相覆盖、语义稀释。也因此它很依赖运行时的 `prompt` 治理——控制面是不断被注入、覆盖、折叠和压缩的动态组合物,而非静止法规。这与 `loop` 性格相配:“每轮都要重新计算”“现在什么最重要”。

2.3 Codex 的控制面是带编号的公文系统

Codex 正相反——它不愿让 `instruction` 只是“模型自己体会”的自由文本。`ContextualUserFragmentDefinition` 这命名已相当直白:强调片段类型、起止边界、包裹规则和消息转换。一段本地规则不只是“有内容”,还必须能被识别为某一类内容:`AGENTS.md` 是明确的 `fragment`, `skill` 是明确包裹过的上下文单元。

这不是概念层面的漂亮命名。`fragment.rs` 把 `AGENTS_MD_START_MARKER` / `AGENTS_MD_END_MARKER` / `SKILL_OPEN_TAG` / `SKILL_CLOSE_TAG` 定成常量,由 `ContextualUserFragmentDefinition::wrap()` 和 `into_message()` 包装成 `ResponseItem::Message`。`user_instructions.rs` 里 `UserInstructions` 把目录序列化成 `# AGENTS.md instructions for ...`, `SkillInstructions` 还带

<name> 和 <path>。Codex 连”这段规则来自哪个目录、哪个 skill 文件”都不让模型猜。

直接后果：控制面可调试性更强——哪段内容来自哪里一目了然；也更适合继续程序化——今天是 marker，明天就可能细化为 precedence、merge rule 或 visibility rule。

骨架：两种控制面装配 (skeleton)

```
// 骨架：Claude Code 动态拼装 (源：constants/prompts.ts, utils/systemPrompt.ts)
system_prompt = concat(
  default_prompt,          // 底板
  append_prompt,           // 外加
  agent_prompt,            // 角色
  claudemd_layers,         // team / personal / project
  memory_sections,         // session memory
  output_style             // 表达约束
)

// 运行时每轮重算：memory 预取、collapse、microcompact、autocompact

// 骨架：Codex fragment 拼装 (源：instructions/src/fragment.rs, user_instructions)
for frag in [agents_md, skill, user_instructions]:
  body = ContextualUserFragmentDefinition::wrap(
    START_MARKER, content, END_MARKER,
    meta { source_dir, name, path }
  )
  msg = frag.into_message()          // -> ResponseItem::Message
  thread.append(msg)
```

不变式 (invariants)

```
assert every fragment has matching (START_MARKER, END_MARKER) # marker 成对
assert fragment.source ∈ {AGENTS_MD, SKILL, USER}           # 类型可识别
assert precedence(project) > precedence(team) > precedence(default) # 优先级
assert claudemd_layers 覆盖顺序 = team → personal → project    # 后注入覆盖前注
assert child_agents_md enabled ⇒ 附加 scope/precedence 说明    # 作用域显式
```

2.4 AGENTS.md 与 CLAUDE.md：同样是本地规则，气质却不同

CLAUDE.md 更接近工程现场公告板：贴近任务目录、和 memory/skill 一起构成“做事时该记住什么”，适合登记常识、禁忌与局部制度。AGENTS.md 则被 Codex 拉进 hierarchy 讨论——docs/agents_md.md 明确说，即便当前没有 AGENTS.md，开启 child_agents_md 时系统也会追加作用域与优先级说明。Codex 关心的不只是“有没有规则”，还包括“规则的适用范围和继承关系有没有被系统明说”。

一句话：Claude Code 让现场规则进入会话，Codex 让现场规则进入制度。

2.5 两种控制面的代价

运行时装配线灵活但难以彻底形式化，依赖主循环和工程经验，规则一多就要防互相覆盖和语义稀释。结构化片段更显式但更重：marker、类型、序列化和注入方式都要定义，还得区分谁是一等对象。前者养出经验型控制力，后者养出制度型控制力——一个长于灵活但不够显式，一个长于清楚但要持续维护结构成本。

2.6 这章的比较结论

Claude Code 把 prompt 视为运行时控制面的动态拼装结果，Codex 把 instruction 视为可识别、可包裹、可序列化的结构化片段。

前者更像现场编导，后者更像制度秘书。谁更先进这问题没意义——真正要看的是系统更怕哪种失控：怕长会话指令失真、现场变化太快，欣赏 Claude Code 的动态装配；怕规则来源不清、作用域模糊、无法系统化治理，欣赏 Codex 的 fragment 化。下一章谈更深的分野：连续性究竟寄托在 query loop，还是寄托在 thread、rollout 和 state 这些更显式的结构上。

第 3 章心跳放在哪：Query Loop 对照 Thread、Rollout 与 State

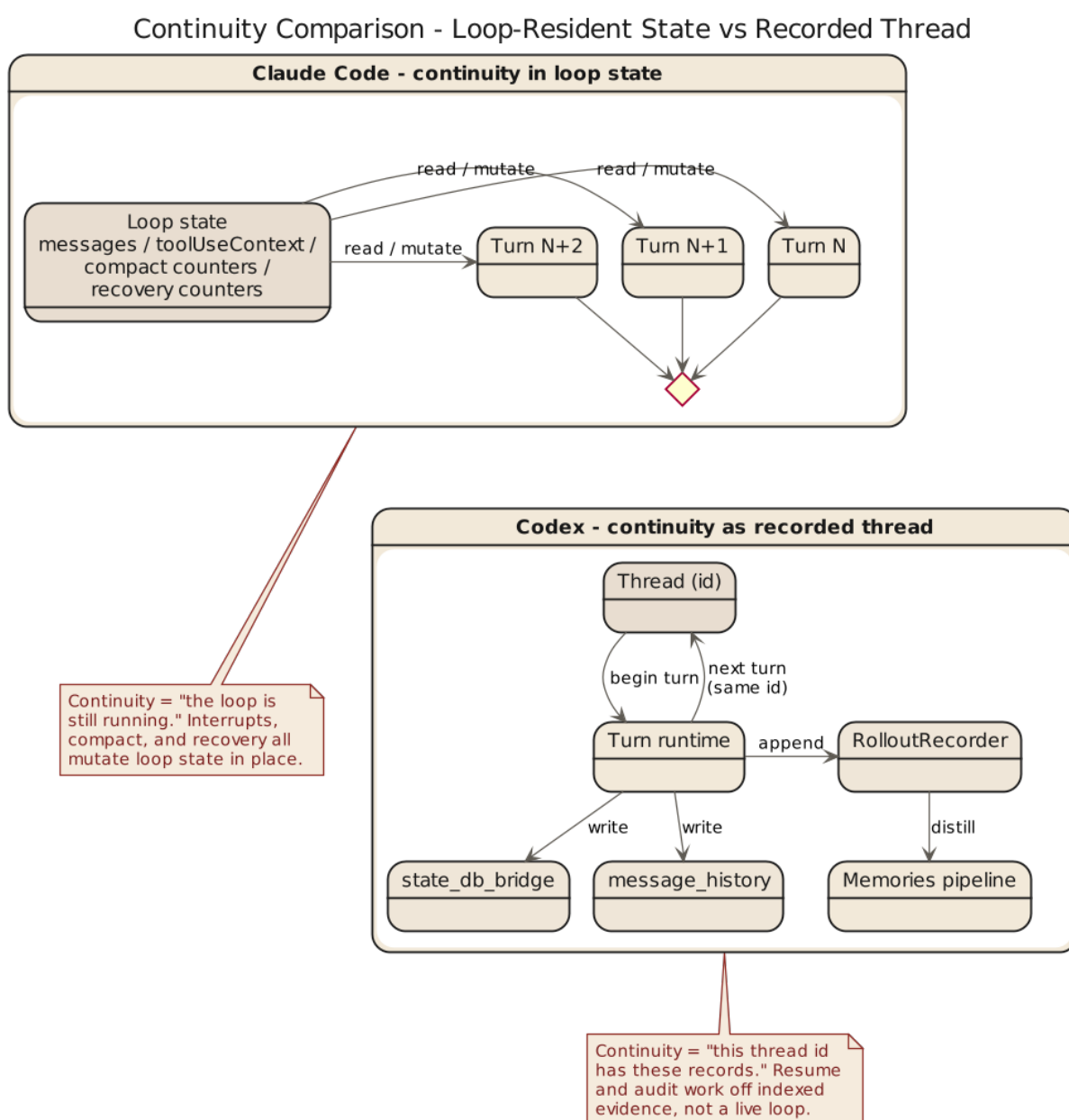


图 2: 连续性对比图

3.1 代理系统的核心是连续性

把代理系统当成“多轮聊天”，就像把数据库当成“一个比较耐心的记事本”——掩盖了真正的架构难题。代理系统真正难的是连续性：上一轮做了什么这一轮怎么接、工具结果怎么回填、中断后怎么收口、上下文过长怎么整理、失败时要重试还是忠实汇报。这些问题决定系统是不是 agent，而不只是某个“支持工具调用的问答接口”。Claude Code 与 Codex 在这里的差异，比任何功能对照都更有含金量。

3.2 Claude Code：把连续性压进主循环

Claude Code 的主轴就在 `query()` 和 `queryLoop()` 一带。它把很多关键问题都压进循环状态里：当前消息序列、tool use context、compact 跟踪、output token 恢复计数、pending summary、turn count、transition。也就是说，对“代理如何活着”的回答是运行时性的——连续性主要由 loop 维护，系统骨架是一个不断自我校正的会话发动机，而非先有强外部状态模型再驱动执行的体系。优势很现实：工具返回顺序、输出截断、prompt too long、history snip、microcompact、用户插话，这些真实麻烦都发生在 loop 里。它不回避这些问题，而把它们当作 loop 内部的合法状态。设计有工程的粗粝感——不优雅但通常更稳。

3.3 Codex：把连续性拆成线程、rollout 与状态桥

Codex 更“账本化”。`core/src/lib.rs` 里连续性被分摊到 `codex_thread`、`thread_manager`、`rollout`、`state_db_bridge`、`state`、`message_history`。

`sdk/typescript/src/thread.ts` 里 `thread` 已经是外部开发者可直接操作的一级概念：`Thread` 持 `id`，可 `runStreamed()` 或 `run()`，`thread.started` 回填线程 ID；`approval policy`、`working directory`、`sandbox mode`、`network access`、`additional directories` 等 turn 级条件都是紧耦合显式参数。“线程主权”很具体——`runStreamedInternal()` 先做 `normalizeInput()` 分离文本和图像，调 `createOutputSchemaFile()` 备 schema，再把 `threadId`、`approvalPolicy`、`sandboxMode`、`workingDirectory`、`networkAccessEnabled`、`additionalDirectories` 一并送进 `_exec.run()`。收到 `thread.started` 事件，对象即更新 `_id`。`thread` 不是外围包装，是 turn 级执行语义真正经过的一层。连续性不再只是“循环还在继续”，而是“一个线程被一套显式状态结构持续记录和约束”。rollout 的存在尤其

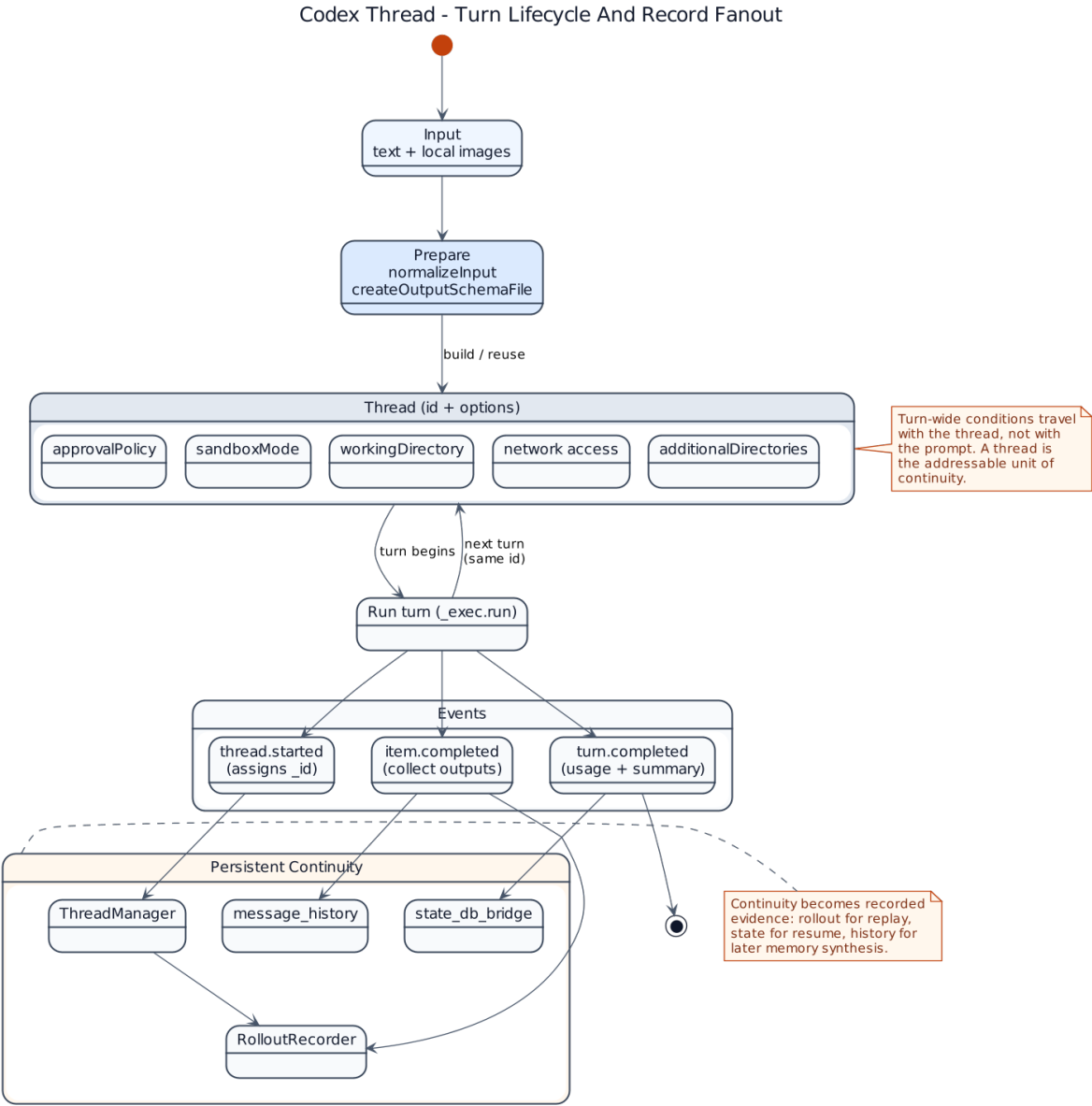


图 3: Codex 线程、turn 与状态细节图

说明 Codex 在乎回放、索引、持久化和会话外可见性——系统因此更接近执行记录器，而非现场对话管理器。

3.4 差别在于状态安放的位置

Claude Code 当然有状态，Codex 当然有循环——差异不在有没有，而在主权在哪。Claude Code 把状态主权更多交给 query loop：系统认为”会话怎么继续”是 runtime 核心问题，许多事情要在 loop 里直接解决。Codex 把状态主权更明确地交给 thread 和 rollout 结构：连续性不该是内部控制流的副产物，而是被线程和状态基础设施承接的显式事实。因此 Thread 在 TypeScript SDK 里已是产品概念，开发者直接围绕它思考 agent turn；Claude Code 的 query loop 则更像发动机室，重要但不一定是用户心智模型的中心。

不变式：连续性主权 (invariants)

```
// Claude Code (源：src/query.ts)
assert loop owns {messages, toolUseContext, compactTracking, turnCount}
assert every loop iteration recomputes "what matters now"
assert pending tool_use 在同一 loop iteration 内必须被闭合或 synthetic 兜底

// Codex (源：core/src/lib.rs, sdk/typescript/src/thread.ts)
assert thread.id is stable across runs; rollout records every turn
assert turn-level {approvalPolicy, sandboxMode, workingDirectory} 在 exec.run
assert thread.started 事件先于任何工具调用 emitted
assert state_db_bridge 持久化早于主循环退出
```

中断与 pending tool_use 的故障矩阵

事件顺序	前置状态	触发	Claude Code	
			下一步	Codex 下一步
用户中断 + 工具在飞	tool_use 未闭合	用户 abort	在 loop 内 synthesize tool_result, 闭账	线程层 abort, 写 rollout interrupted 事件

事件顺序	前置状态	触发	Claude Code	
			下一步	Codex 下一步
模型输出被截断	max_output_tokens 上限触发	token 触发	提升 cap 或追加 meta user msg 续写	线程记录 truncation, 由 caller 重启 turn
prompt too long	会话过胀	prompt_too_long	loop 内 collapse / reactive compact / surface	由 thread_manager 裁剪 history 再下一轮
进程重启	会话中途崩溃	crash / kill	依赖外部 PR / Git 重建, loop 状态易丢	rollout + state_db_bridge 可按 thread.id 重放
恢复失败	连续 compact 失败	熔断阈值	跳过 stop hook, faithful 报错	返回 state bridge 错误, 保留 thread 记录

3.5 对恢复与可审计性的影响

状态安放差异直接影响恢复和审计。Claude Code 的恢复强项在离现场近——很多问题就在 loop 内部被发现和修复 (reactive compact、token 上限恢复、工具中断处理), 不需要先搬运到更高一层状态模型里再考虑怎么回滚。Codex 的恢复强项体现在状态可追踪性——线程有 ID, rollout 有记录, state bridge 和 message history 提供更清楚的外部结构, 系统更容易回答”上一轮到底发生了什么”。core/src/lib.rs 的”档案意识”更明显: 不只暴露 CodexThread, 还显式导出 ThreadManager、RolloutRecorder、state_db_bridge 和 message_history——一个系统把这些器官放到根模块出口附近, 等于承认连续性不是 loop 的副产品而是基础设施本身。简言之: Claude Code 像现场救火队, Codex 像带档案系统的调度中心; 前者擅长维持运行, 后者擅长说清楚运行是如何被维持的。

3.6 对产品和团队接口的影响

主权在 loop，团队沿运行时问题组织工作：哪些错误需要恢复、哪些动作应当中断、compact 何时触发、工具结果如何串回主对话。主权在线程和状态结构，团队沿接口与治理组织工作：thread 的生命周期、rollout 保留哪些事件、状态库放哪、approval policy 如何成为 turn 级选项。因此 Claude Code 更像先把 agent 做会再嵌制度，Codex 更像先立制度接口再让 agent 在里面工作。

3.7 本章结论

这一章的结论可以写得明确一点：

Claude Code 的连续性更多由 query loop 承担，Codex 的连续性更多由 thread、rollout 与 state 基础设施承担。

前者强调 runtime heartbeat。

后者强调 persisted session substrate。

这不是审美差异，这是系统权力分配。谁来拥有连续性，谁就定义了 harness 的中心。

下一章进入最硬的一层：工具、沙箱、审批和执行策略。到了这里，浪漫叙事一般都会自动退场，因为 shell 不大关心文风。

第 4 章工具、沙箱与策略语言：谁来阻止模型动手太快

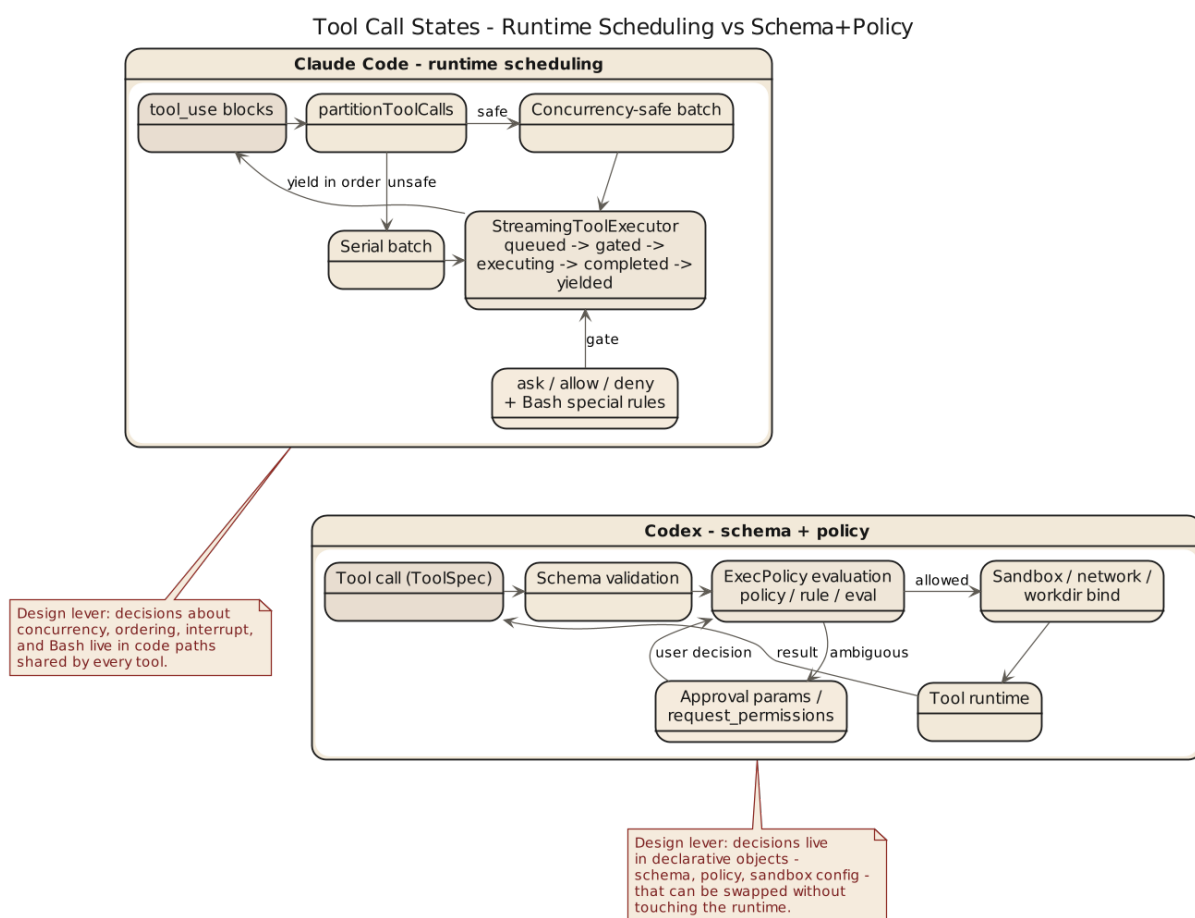


图 4: 工具治理对比图

4.1 真正危险的是开始执行

模型说错话只浪费时间，跑错命令就可能把目录、仓库、进程和工作流一起带坏。真正把 AI coding system 区分开来的，是调工具之前谁拥有最后解释权。Claude

Code 和 Codex 都很认真，但认真得不一样：前者在运行时把工具纳入调度纪律——`toolOrchestration.ts`、`toolExecution.ts`、`StreamingToolExecutor.ts`、`useCanUseTool.tsx`、Bash 专门 prompt 和 `allow/deny/ask` 语义，关心的是能不能跑、怎么跑、能不能并发、用户有没有拒绝、如何中断、结果怎么回到上下文。后者把工具本身做成类型化接口——`tools/src/lib.rs` 导出工具构造器，`local_tool.rs` 定义 `exec_command`、`shell`、`shell_command`、`request_permissions` 等 schema。工具在 Codex 里首先是规范化 API，其次才是执行单元。

4.2 Claude Code：重点在运行时编排和危险动作约束

Claude Code 的工具系统有强烈现场调度感：并发看 `schema` 和 `isConcurrencySafe()`，上下文修改要保证回放顺序稳定，流式执行还要处理中断、`synthetic result` 和 UI 反馈。它承认工具调用是“一个带后果的过程”而非单点动作——像给模型装工地监工：先做哪个、哪些能并行、哪些串行、做完怎么记账、中途被叫停怎么办。对 Bash 这类高风险工具的唠叨，正是成熟工程对危险接口的态度。

4.3 Codex：重点在工具 schema、审批参数和策略引擎

Codex 把“风险动作”的控制做成正式接口约束。`local_tool.rs` 里 `exec_command` 显式拥有 `cmd`、`workdir`、`shell`、`tty`、`yield_time_ms`、`max_output_tokens`、`login` 和 `approval` 相关字段——不是接收一个字符串命令了事；`shell / shell_command` 描述层直接要求设 `workdir`、提醒别滥用 `cd`。正确使用方式被嵌进工具定义本身。审批和权限提升抽成显式参数，`request_permissions` 做成单独工具，`execpolicy` 单独做成 crate——`Policy`、`Rule`、`Evaluation`、`Decision`、`parser` 这套命名等于说：执行边界已经是一门小型政策语言，不只是几个 `if/else`。而这不是虚张：`schema required` 字段明确、`additional_properties` 关闭；`execpolicy/src/lib.rs` 不仅导出 `parser` 和 `PolicyParser`，还导出 `blocking_append_allow_prefix_rule`、`blocking_append_network_rule` 这种修正 helper——Codex 不只检查政策，还准备了修改和补丁的正式入口。

4.4 运行时审批对照策略语言

Claude Code 偏运行时审批链，Codex 偏显式策略语言与参数化审批。Claude Code 的 `ask/allow/deny` 与调用现场紧密耦合，按上下文、工具类型、用户动作和会话状态

决定，长处是灵敏，缺点是规则容易藏在 runtime 逻辑里。Codex 的 exec policy 把规则抽离，可单独解析和评估，长处是可读性和可迁移性更强，适合团队治理，缺点是系统更重，policy 设计必须当正事维护。直说：前者像”值班经理现场拍板”，后者像”公司先写好制度，再看这单是否合规”。

骨架：两种风险闸门 (skeleton)

```
// 骨架：Claude Code 运行时审批 (源：src/hooks/useCanUseTool.tsx, StreamingTool)
decision = hasPermissionsToUseTool(tool, input, ctx) // allow | deny | ask
match decision:
    allow: schedule_with_concurrency_check(tool)      // isConcurrencySafe()
    deny:  reject(reason)                             // sticky
    ask:   route_to(coordinator | swarm | interactive)
interrupt_semantics = tool.interruptBehavior ∈ {cancel, block}

// 骨架：Codex exec-policy 评估 (源：execpolicy/src/lib.rs, local_tool.rs)
policy = PolicyParser.parse(source)
for rule in policy.rules:
    eval = rule.evaluate(exec_command { cmd, workdir, shell, tty,
                                     yield_time_ms, max_output_tokens, login })
    if eval.matches: return Decision::{Allow | Deny | RequestPermissions}
return Decision::default
```

审批决策树

```
tool_call
├─ schema 校验失败          → deny (additional_properties=false 阻止乱塞)
├─ 高危前缀匹配 deny rule   → deny
├─ 网络/提权 匹配 ask rule   → request_permissions (显式工具)
├─ 无匹配 & sandbox 可覆盖   → allow (受 workdir/sandbox mode 约束)
└─ 无匹配 & sandbox 保守     → ask
```

超时与参数阈值

名称	含义	源引用
yield_time_ms	单次 exec 允许阻塞的毫秒上限	local_tool.rs (exec_command)
max_output_tokens	工具输出纳入上下文的 token 上限	local_tool.rs
additional_properties=rules	防止模型乱塞参数	local_tool.rs (schema)
Bash 子命令数 cap	单次 Bash 调用允许的复合子命令上限	bashPermissions.ts

4.5 沙箱与审批：产品定义问题

沙箱、审批、权限不是安全附属件——对 coding agent 而言，它们定义了产品是什么。系统若允许模型在用户目录直接跑任意命令，就把风险转嫁给了用户；反过来，能显式表达 sandbox mode、network access、approval policy、additional directories、state DB 位置和 MCP tool approvals，才是同时交付能力与行为边界。Codex 把 turn 级条件暴露在 thread.ts 里，视之为线程语义；Claude Code 把边界压到运行时现场——工具执行、中断、permission hook 与 Bash 限制。一个”边做边看守”，一个”先给出执行契约再开始做”。

4.6 MCP、扩展工具与边界外移

两套系统都能接外部能力，但差异仍在。Claude Code 把 skill / hook / permission / 工具 prompt 拼成场景化治理链，让本地规则跟着现场进主循环；Codex 则把外部能力统一纳入工具系统——tools/src/lib.rs 的 MCP resource、dynamic tool、tool discovery 说明扩展也应是 schema 化、公理化的工具对象，而非临时约定。生态一旦变大，“扩展能力如何服从总规则”就成了压舱石：谁先想清楚边界外移，谁的扩展体系就不会变成杂物间。

4.7 本章结论

这一章可以压缩成一句比较硬的判断：

Claude Code 的工具治理更强地依赖运行时编排与现场审批，Codex 的工具治理更强地依赖 schema、参数化权限和独立策略系统。

前者像经验丰富的工头。

后者像有制度处和法务部的施工单位。

你要是只看“都能跑命令”，就会错过真正重要的差异。重要的是，谁在工具动手之前拥有最终秩序。

下一章看更接地气的一层：skills、hooks、本地规则文件和团队制度。技术系统一旦要进团队，最后都得学会写村规民约。

第 5 章技能、Hook 与本地规则：系统如何学会守乡约

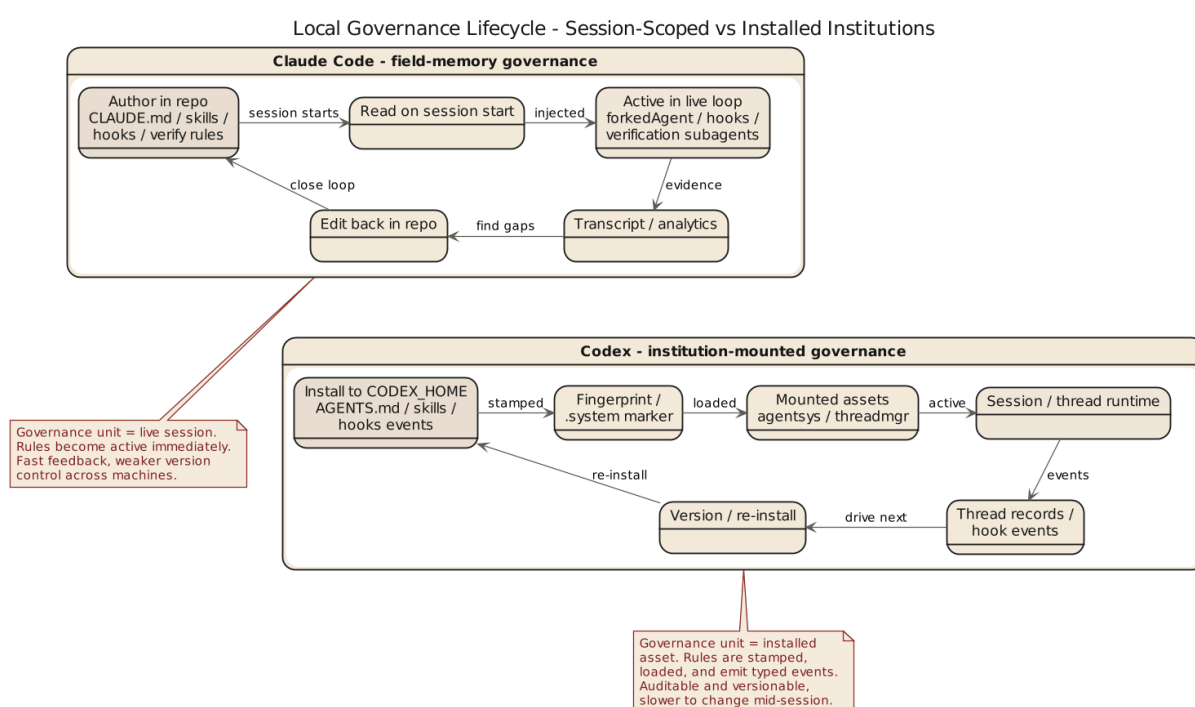


图 5: 本地治理对比图

5.1 真正能落地的 agent，一定会地方化

任何通用 coding agent 一旦开始给团队干活，就遇到同一问题：公司有公司规矩，仓库有仓库规矩，目录有目录规矩，人还有怪脾气。系统不能吸收这些局部制度，就只能停在演示环境里。Claude Code 和 Codex 都给出答案，方向不同。

5.2 Claude Code：把局部制度做成现场记忆

地方化能力落在 CLAUDE.md、skill、hook、session memory——组合起来有”现场经验沉淀”味道：CLAUDE.md 说清此仓库/目录/团队的常识，skill 打包 workflows，hook 挂生命周期，session memory 让当前工作不必每轮重新做人。共同特点是贴近任务现场——让规则进入当前会话、参与当前执行，而不是先定义万古不变的制度。像一个随身带笔记本的工程师，走到哪把当地规矩抄下来：很实用，适合多项目多目录多约束并存，但没额外整理的话知识容易以”现场补丁”扩张。

5.3 Codex：把局部制度做成结构化注入和事件系统

Codex 也有 skill、本地规则、hook，但更制度化。skill: skills/src/lib.rs 把 system skills 装到 CODEX_HOME/skills/.system 并做 fingerprint——skill 不是临时读入的文本，而是被安装、被管理、可追踪版本的资产。install_system_skills() 比对 fingerprint，仅 marker 不匹配时才覆写。AGENTS.md 不只是”读一份本地说明”，还伴随作用域和 hierarchy 讨论——局部规则不只是内容，还带位置关系。hook: hooks/src/engine/mod.rs 把事件拆成 session_start、pre_tool_use、post_tool_use、user_prompt_submit、stop，每个 handler 都有 event_name、matcher、timeout、status message、source path、display order——接近显式生命周期事件系统。engine 还分 preview_* / run_* 两条路径（先预览再执行），Windows 上因能力不完整明确关闭 codex_hooks 并返回 warning——hook 能不能开、为什么不开，都希望可解释。

5.4 Claude Code 偏经验收编，Codex 偏制度挂载

Claude Code 的本地治理把现场经验不断收编进主循环附近，擅长让 agent 在当前上下文里迅速学会”这里怎么办”；Codex 把地方规则挂载到明确控制面与生命周期机制上，擅长让规则被分类、排序、安装、触发。团队感也就不同：前者像熟悉现场、懂看气氛的老员工，后者像制度意识极强的新项目经理——先把规则贴出来再协调人做事。

骨架：Codex hook 生命周期 (skeleton)

```
// 骨架：Codex hook engine （源：hooks/src/engine/mod.rs）
events = [session_start, user_prompt_submit, pre_tool_use, post_tool_use, stop]
```

```

for ev in events:
    handlers = preview_handlers(ev, ctx)           // 先预览命中
    emit(preview_event { ev, handlers })
    for h in handlers sorted by display_order:
        if match(h.matcher, ctx) and not timed_out(h.timeout):
            run_handler(h)                         // 真正触发
        else:
            log_skip(h, reason)
on platform == windows: disable(codex_hooks); warn("incomplete support")

```

不变式：hook 事件顺序

```

assert session_start fires once per thread before any tool_use
assert pre_tool_use fires immediately before execution; post_tool_use after
assert stop fires exactly once per thread termination path
assert preview_* path never executes handlers; only run_* does
assert each handler has {event_name, matcher, timeout, source_path, display_order}
assert display_order 稳定 ⇒ 同一事件下多 handler 可重放
assert skill fingerprint 不匹配 ⇒ 触发重装；匹配则跳过 (skills/src/lib.rs)

```

5.5 对组织可复制性的影响

主要靠现场经验注入的系统，适应新仓库更快、在复杂局部语境中更有效；但复制到更多团队时需要额外整理，否则各写各的 CLAUDE.md、各做各的 skill，像各省自印教材。主要靠结构化注入和事件挂载的系统，在组织扩展上更有潜力——规则易统一分发、版本化和审计；代价是学习成本更高，团队要先接受更多显式制度。经典取舍：越贴近现场越有弹性，越制度化越易复制。真正决定结果的，是团队需要哪一种稳定性。

5.6 本章结论

这一章的归纳可以写成：

Claude Code 更倾向于把局部治理做成现场记忆与运行时注入，Codex 更倾向于把局部治理做成结构化资产与生命周期事件系统。

这不是“都支持 skills 和 hooks”的同义反复。

差别在于，Claude Code 问的是“怎样让 agent 在这里干活更像本地人”，Codex 问的是“怎样让本地规则进入一套可管理的制度框架”。

下一章要看这两种系统在更高风险处如何分工：多代理、验证、持久状态和恢复。系统一旦开始让多个代理干活，光讲规矩还不够，还得讲责任分离。

第 6 章委派、验证与持久状态：谁来防止系统自己给自己打高分

6.1 多代理的真正问题是责任

一听多代理就想到效率提升，像听公司扩编——其实真正棘手的从来是多出来的责任怎么切。若同一系统既执行又总结又验证，还顺手给自己写评语，结论多半令人宽慰但不可靠：“干得不错”。Claude Code 清醒地把 explore、execute、synthesis、verification 拆开，verify 是独立纪律而非礼貌动作——系统不愿让”完成”由执行代理自己宣布。Codex 也走这条路：tools/src/lib.rs 里 `create_spawn_agent_tool_v*`、`create_wait_agent_tool_v*`、`create_send_message_tool`、`create_close_agent_tool_v*` 说明委派是正式工具能力，不是黑魔法。

6.2 Claude Code：多代理服务于运行时职责分区

Claude Code 的多代理围绕主循环和任务推进展开——主代理不该什么都自己干，尤其不该既干活又验收。因此多代理主要用于：探索外包、执行分流、synthesis 汇总、verification 独立复核。这很符合它的强项在运行时编排——多代理被纳入”当前这轮怎么推进”的治理框架，而不是先有 agent platform 再往里塞任务。

6.3 Codex：多代理服务于显式工具化协作

Codex 把委派定义成工具接口，多代理更像正式子系统。两个直接影响：委派动作更易被记录、审计和组合（显式工具调用，不是 runtime 魔法）；协作更易与线程、状态和审批体系对齐（thread/rollout/policy 本就一等公民）。agent_tool.rs 里 `spawn_agent`、`send_input`、`wait_agent`、`close_agent` 各有 schema: `send_input`

区分 `interrupt=true` 立即打断和默认排队; `wait_agent` 有 `default/min/max timeout`; `close_agent` 连 `open descendants` 一起关闭——抢占、等待、收尾都是协议字段。适合做成平台能力，不见得更灵巧但更易长期维护。

6.4 持久状态让验证不只是礼仪

验证流于形式的一个主因是系统没有足够好的状态承接——上一步干了什么、为什么、哪些工具动过、哪些文件变过，若只在执行代理脑子里，验证就容易沦为貌似认真实则缺材料的表演。Claude Code 让会话状态、工具结果和恢复分支在 `runtime` 里连续可见，配合独立验证纪律降低自我美化。Codex 通过 `thread`、`rollout`、`message history`、`state DB bridge` 给验证提供更清楚的材料基础。二者并不冲突——前者补执行者过于沉浸现场的问题，后者补系统协作必须留下结构化证据的问题。

6.5 对恢复与收尾的不同态度

Claude Code 很在乎 `task cleanup`、父子 `abort` 传播、`subagent lifecycle hook`——在它的世界里，多代理首先是运行时现场的一部分，出问题必须能及时收口。Codex 则把代理生命周期纳入显式状态管理和调用协议：不只关心“子代理死没死”，还关心“委派行为作为系统事件该如何留存”。前者像现场总工，担心人散场后地上还留坑；后者像带项目管理系统的组织者，担心每个协作动作是否进入记录体系。

骨架：Codex 代理委派协议 (skeleton)

```
// 骨架：agent_tool.rs spawn/wait/send/close
handle = spawn_agent { role, prompt, timeout, inherit_approval }
for msg in updates:
    send_input(handle, msg, interrupt ∈ {true, false}) // true=立即打断, false=排队
result = wait_agent(handle, timeout ∈ [min, default, max])
close_agent(handle, cascade=true) // 连同 open descendants 一起关闭
```

孤儿与超时故障矩阵

事件顺序	前置状态	触发	下一步	阈值
父代理 abort	子代理在飞	parent.abort	cascade abort 传到 handle; 写 rollout 事件	—
wait_agent 超时	子代理未返回	timeout ≥ max	关闭 handle, 返回 timeout 结果	wait_agent.max
send_input(interrupt=true)	子代理排队中	抢占	丢弃队列, 注 入新输入	—
close_agent	有 open descendants	显式关闭	级联关闭所有 后代	cascade=true
子代理崩溃	—	异常退出	返回 error, 保 留 thread 记录	—
handle 泄漏	任务结束未 close	finalize	强制 close + evict	不允许悬挂句 柄

6.6 本章结论

这一章的结论不难写：

Claude Code 的多代理设计更强调运行时职责分离与现场收尾，Codex 的多代理设计更强调工具化委派、状态承接与可审计协作。

二者都试图避免系统自己给自己打高分。

只是 Claude Code 更靠角色分离和验证纪律。

Codex 更靠显式接口、线程状态和协作记录。

最后一章，我们把前面六章压成总判断，回答书名里的问题：究竟是殊途同归，还是根本不同种。

第 7 章殊途同归，还是各表一枝

7.1 先说“同归”的部分

要是只问结论，我会先说：它们确实殊途同归。

原因很简单。Claude Code 和 Codex 都不把模型当作值得直接托付的执行体。它们都承认：

- prompt 不等于控制全部
- 工具必须受约束
- 长会话一定需要状态治理
- 本地规则必须进入系统
- 多代理必须有分工和验证

换句话说，它们都已经越过了那种“只要模型更强，系统问题就会自动消失”的幼稚阶段。谁能走到这一步，谁就已经不再把 agent 当成聊天机器人带几把工具那么简单。

所以从总体方向看，它们确实在同一个目的地上会合：把 harness 当成真正的控制层，把模型当成这层控制之下最不稳定、但也最有生产力的部件。

7.2 再说“各表一枝”的部分

但要是因此说它们本质一样，那就太粗糙了。

Claude Code 的主轴更像这样：

- 从 query loop 出发
- 在运行时处理连续性
- 用 compact、工具编排、中断和恢复维持秩序
- 用技能、hook、验证把现场规则和团队制度接进来

Codex 的主轴更像这样：

- 从模块边界和控制层显式化出发
- 把 instruction 做成 fragment
- 把工具做成 schema
- 把执行边界做成 policy
- 把会话做成 thread / rollout / state
- 把本地规则与 hook 做成结构化资产和事件系统

前者像从机械经验里塑出来的系统。

后者像从制度设计里塑出来的系统。

这就是“各表一枝”的地方。差别不在目的，而在骨架。

7.3 如果非要给它们起一个更难听但更准确的名字

我甚至愿意把它们叫作两种不同的 harness 政体。

Claude Code 比较接近运行时共和制。很多权力集中在主循环和现场调度上，秩序通过不断的现实协商来维持。它并不是不讲制度，只是制度往往服务于会话现场。

Codex 比较像控制面立宪制。很多权力首先被写进类型、片段、策略、线程和事件系统里，运行时当然还要做判断，但那套判断更倾向于在明确框架下展开。

这么说当然有点夸张，不过有助于看清一件事：harness 从来不只是技术组件堆积，它也是一种权力分配方式。谁定义边界，谁解释状态，谁拥有最后的执行解释权——这些事情最后都会体现在架构里。

7.4 对后来者的启发

如果团队准备做自己的 coding agent，这份比较真正有用的地方，是帮你少犯两类错误，而不是让你选边站。

第一类错误，是误以为只要学一套功能表就够了。其实不够。你得先决定自己的主要矛盾是什么。是长会话里容易失控，还是规则来源太散、权限边界不清？不同矛盾，会把你推向不同 harness 形态。

第二类错误，是试图把两者最顺眼的特性不加判断地拼在一起。工程上最危险的往往不是取舍，而在没有取舍。你既想要完全动态的运行时灵活性，又想要彻底显式的结构化控制面，最后多半会得到两头都不彻底的系统。

更合理的做法是：

- 如果你更怕现场失控，就先把 runtime heartbeat 做扎实
- 如果你更怕制度失真，就先把 instruction、tool、policy、state 显式化
- 等主矛盾稳住，再逐步补另一边

这里还可以多加一个提醒。现实里很多后起系统既没有把运行时纪律做扎实，也没有把控制层显式化到位，而是走向第三种更容易上手、也更容易失控的路线：把越来越多的 bootstrap 文件、角色设定、技能说明和工作区文本堆进 prompt，试图靠“信息更全”弥补骨架不足。这类系统短期里常常显得能跑，长期里却容易同时暴露两类问题：token 烧得快，工作语义又不够稳。

7.5 最终判断

书名里的问题，现在可以正式回答。

它们是殊途同归。

也是各表一枝。

“同归”说的是最终承认的现实一致：模型不可靠，harness 才是秩序来源。

“各表一枝”说的是实现这个现实的政治经济学不同：Claude Code 更信运行时纪律，Codex 更信显式控制层。

而那些主要靠 prompt 堆叠来维持上下文的系统，则更像还停在中间地带。它们已经意识到模型会忘、会漂移，所以开始加 memory、加 skills、加 compact；但如果上下文治理仍然主要依赖“先注入，再抢救”，那就还没有真正决定把秩序放在哪一层。

这两种路径没有谁天然更高明。真正的问题是，你的系统准备把不确定性关进哪一层笼子里。

笼子的位置，决定了系统以后会演化成什么样。

第 8 章如果你要自己做：该向谁学，先学什么

8.1 比较的最终用途是少走弯路

最没意思的比较文结尾是把读者推回消费主义姿态——二选一。工程系统不是耳机，不靠横评下单。真正有用的问题是：自己做一套 harness 或重构现有 agent，先学谁的哪一部分。Claude Code 和 Codex 给出的是两种起手式，不是两份答案。Claude Code 提醒你：别把运行时问题想得太文雅——真正拖垮系统的是 query loop 里的脏活（工具结果收口、上下文膨胀、中断恢复、子代理清理、验证独立、失败熔断），轻视它们就会得到一个看着聪明、用着费命的系统。Codex 提醒你：别把控制层做成一团心照不宣——instruction 来源、tool schema、approval policy、thread state、hook event、skills 资产越早显式化越好治理，指望运行时临场发挥代替制度的系统，最后都像口头约定搭起来的棚子。

8.2 三种常见团队，三种起手方向

第一种：已经有 agent 原型，但长会话经常失控

这种团队通常最需要先学 Claude Code。

因为他们的问题多半不在“控制面定义不清”，而在系统活不长。常见症状包括：

- 上下文越来越乱
- 工具调用链断裂
- 中断后状态说不清
- 子代理跑完以后没人收口
- 验证变成随口一说

这时候最该补的是主循环纪律，而不是更多配置项。先把 runtime heartbeat 做稳，再谈制度美学。

第二种：已经有不少规则，但规则来源过散、权限边界不清

这种团队通常更该先学 Codex。

因为他们的问题不在现场活不过去，而在系统越来越难治理。常见症状包括：

- 本地规则四处散落
- 哪些约束进了 prompt，哪些进了工具，没人说得清
- 审批逻辑混在代码里，难以解释
- 多种扩展能力接进来以后，边界越来越模糊

这时候最需要的，是把控制层显式化。先把 instruction、tool、policy、thread 这些概念定义清楚，再让 runtime 在里面工作。

第三种：还没有成型系统，准备从零开始

这种团队最危险，因为最容易同时羡慕两边的优点，然后把自己搞成折中失败品。

更稳妥的路线通常是：

- 先选一个主矛盾
- 先围绕主矛盾设计主骨架
- 另一边只补最低限度，不要一口气学全

要是你的第一阶段主要风险是“模型会乱来”，那就先做 Claude Code 式的 runtime discipline。

要是你的第一阶段主要风险是“团队会失去秩序”，那就先做 Codex 式的显式控制层。

最怕的是两边都想一次学全，结果既没有扎实主循环，也没有清楚控制面。

三类团队的分阶段清单 (builder checklists)

类型一：原型已在，长会话失控（先学 Claude Code）

- [] Week 1: 明确主循环状态集 {messages, toolUseContext, compactTracking, tur
- [] Week 1: 每个 tool_use 闭账或 synthetic 兜底；abort 路径落地

- [] Week 2: 上下文治理三件套: memory / collapse / autocompact 阈值表固化
 - [] Week 2: 验证独立于实现 (verifier 不等于 implementer)
 - [] Week 3: 子代理生命周期 SubagentStart/Stop 事件可观测
- Gate: 连续 24h 会话无 token 熔断、无孤儿子代理、无 tool_result 泄漏

类型二: 规则已多, 来源散乱、边界不清 (先学 Codex)

- [] Week 1: 所有 instruction 进入 fragment 化: marker、source、precedence 三
 - [] Week 1: 工具 schema 化, additional_properties=false
 - [] Week 2: approval policy 抽为 rule, deny/ask/allow 可单独评估
 - [] Week 2: thread.id / rollout 建立, turn 级 {approvalPolicy, sandboxMode} 显
 - [] Week 3: hook 事件拆 pre/post/session_start/stop, skill 资产 fingerprint
- Gate: 任何规则变更可通过 PR 直接 diff, 无需运行时改代码

类型三: 从零开始 (选一个主矛盾再动手)

- [] Week 1: 声明主矛盾——"模型乱来" or "团队失序"
 - [] Week 1: 定义最小权限模型 (高危动作 deny/ask 清单)
 - [] Week 2: 主矛盾侧骨架立起 (loop 或 fragment+thread 二选一)
 - [] Week 3: 另一侧只补最小必要 (恢复路径 or 基本 hook)
 - [] Week 4: 引入 1-2 个 skill/工具, 验证闭环
- Gate: 新成员能沿清单推进, 不需要原作者口传

8.3 什么该学 Claude Code, 什么该学 Codex

优先学 Claude Code: query loop 的状态心智、compact 与上下文治理、工具编排与中断处理、子代理生命周期和验证独立、把失败路径当主路径设计。优先学 Codex: instruction fragment 化、tool schema 化、approval/policy 的显式表达、thread/rollout/state 的基础设施化、hook 事件和 skills 资产管理。这不是折衷——前提是你得知道自己在学什么。学习的理由应该是那一块正好补你的短板, 而不是因为别人已经做了。

如果把上一册的上下文治理判断带进来, 再看第三方 harness, 容易识别一种常见但代价很高的路线: 它不把上下文拆成不同寿命、不同职责、不同入口成本的单元, 而是把大量 bootstrap 文件、技能说明、身份设定和工作区文本尽量塞进 prompt, 溢出时靠截断、compact 和恢复链补锅。这类系统表面上也有 memory、skills、compact、上限控制, 治理主轴仍是“先注入, 再抢救”。上下文主要靠堆叠文本组织时, token 浪费只是第一层代价, 更麻烦的是语义信号稀释——模型看到了很多东西, 却不一定更清楚

下一步该抓哪一类工作语义。

三条路线一句话总结：Claude Code 把上下文当工作内存来经营（先想什么该保住、什么该压缩）；Codex 把上下文当结构化单元来治理（先想来源类型、作用域和状态承接）；OpenClaw 这一类则把上下文当 prompt 容器来扩容（先想还能再塞什么，超过了再说）。这也是很多团队起初觉得这种路线“信息更全”，长会话多代理复杂任务时却同时抱怨两件事——token 烧得快、效果没随上下文变胖而稳定提升。它解决的是装进去多少，不是继续工作真正需要保住什么。

8.4 一个危险误区：把”显式”与”灵活”误认为天然对立

做系统的人常有一种偷懒的对立法：一说显式控制层就觉得系统会太重太慢；一说运行时灵活就觉得可以先靠经验做，结构以后再说。都不高明。显式不必然僵硬，灵活也不必然混乱。真正的问题是——你有没有清楚定义”哪些必须显式、哪些可以留给现场判断”。Claude Code 的长处不在排斥结构，而在知道哪些麻烦必须运行时正面处理；Codex 的长处不在排斥灵活，而在知道哪些边界不先讲清楚，后面任何灵活都会变成争议。好的第三种 harness 不折中平均，而明确区分：哪些规则必须先写死、哪些判断留给运行时、哪些状态必须持久化、哪些经验只在会话内暂存。

8.5 给后来者的一组顺序建议

从零开始做一套 harness，推荐这个顺序：(1) 高风险动作和最小权限模型；(2) 主循环或线程生命周期；(3) 上下文治理与恢复路径；(4) 技能、本地规则与 hook；(5) 多代理、平台化和复杂生态。顺序不性感，但大致符合事故发生的顺序。工程里的设计顺序，应该按事故出现的先后排，而不是按演示时的好看程度排。

8.6 本章结论

这一章只想留下一句朴素的话：

学 Claude Code，主要是学如何让系统在现场稳定运行；学 Codex，主要是学如何让系统在组织里长久维持秩序。

谁只学前者，容易变成经验过强、制度不足。

谁只学后者，容易变成制度漂亮、现场发虚。

真正值得做的，是根据自己的主矛盾决定先长哪根骨头，而不是选边站。

附录 A 源码地图：这套比较主要依据哪些文件

这份附录只做一件事：说明各章判断主要基于哪些文件。它不是源码转载目录，也不意味着内容会提供相关源代码副本。

这里仍然保留同样的边界：

- 仅做必要的工程性引用和模块定位
- 不附带 Claude Code 或 Codex 的实现正文
- 不做大段源码转录

A.1 Claude Code 侧主要依据

总体控制面与 prompt：

- `src/constants/prompts.ts`
- `src/utls/systemPrompt.ts`
- `src/utls/claudemd.ts`
- `src/memdir/memdir.ts`

运行时循环与恢复：

- `src/query.ts`
- `src/QueryEngine.ts`
- `src/services/compact/autoCompact.ts`
- `src/services/compact/compact.ts`

工具与权限：

- `src/services/tools/toolOrchestration.ts`
- `src/services/tools/toolExecution.ts`
- `src/services/tools/StreamingToolExecutor.ts`
- `src/hooks/useCanUseTool.tsx`
- `src/tools/BashTool/prompt.ts`
- `src/tools/BashTool/bashPermissions.ts`

多代理、skills、hooks:

- `src/utils/forkedAgent.ts`
- `src/coordinator/coordinatorMode.ts`
- `src/tasks/LocalAgentTask/LocalAgentTask.tsx`
- `src/tools/SkillTool/SkillTool.ts`
- `src/tools/SkillTool/prompt.ts`
- `src/utils/hooks/hooksConfigManager.ts`

A.2 Codex 侧主要依据

核心模块骨架:

- `core/src/lib.rs`
- `tools/src/lib.rs`
- `skills/src/lib.rs`
- `hooks/src/lib.rs`

instruction fragment 与用户注入:

- `instructions/src/lib.rs`
- `instructions/src/fragment.rs`
- `instructions/src/user_instructions.rs`
- `docs/agents_md.md`

工具、审批与执行策略:

- `tools/src/local_tool.rs`
- `tools/src/agent_tool.rs`

- `execpolicy/src/lib.rs`
- `docs/execpolicy.md`
- `docs/sandbox.md`

线程与状态:

- `sdk/typescript/src/thread.ts`
- `core/src/lib.rs`
- `core/src/thread_manager.rs`
- `core/src/rollout.rs`
- `core/src/state_db_bridge.rs`
- `core/src/message_history.rs`

hook 事件引擎:

- `hooks/src/engine/mod.rs`

A.3 各章对照

第 1 章:

- Claude Code 的 `query.ts`、`toolOrchestration.ts`
- Codex 的 `core/src/lib.rs`

第 2 章:

- Claude Code 的 `prompt assembly` 与 `CLAUDE.md`
- Codex 的 `fragment.rs`、`user_instructions.rs`

第 3 章:

- Claude Code 的 `query loop / QueryEngine`
- Codex 的 `thread.ts`、`thread_manager`、`rollout`、`state_db_bridge` 暴露模块

第 4 章:

- Claude Code 的工具编排、Bash 限制、权限语义
- Codex 的 `tools/src/lib.rs`、`local_tool.rs`、`execpolicy/src/lib.rs`

第 5 章:

- Claude Code 的 skill / hook / memory 体系
- Codex 的 skills 体系、hooks engine

第 6 章:

- Claude Code 的 forked agent / verification 纪律
- Codex 的 agent tool 集合、thread / rollout / state

第 7 章:

- 综合前述所有文件形成总判断

附录 B 检查清单：如何判断你的 Harness 更像 Claude Code、Codex， 还是半成品

比较如果不能落成检查清单，最后很容易只剩一些表述完整却难落地的判断。下面这份附录，就是把前面几章压成团队可直接讨论的清单。

B.1 控制面清单 (invariants)

assert every instruction has {source, type, precedence} # fragment 可识别
assert prompt 中 control-plane 与 output-style 明确区分 # 文风不等于秩序
assert 本地规则的作用域 (CLAUDE.md / AGENTS.md) 显式标注
assert team 规则变更可通过 diff 追踪 # 非口头约定

答不清这些，说明控制面还停留在“能用就行”。

B.2 连续性清单 (invariants)

assert 连续性主权 $\in$ {main loop, thread+rollout+state} # 明确归属
assert 中断 $\Rightarrow$ tool_result 同步闭账 (synthetic 兜底也算)
assert 长会话有 compact / truncation / recovery 三件套
assert thread.id / 会话索引 / 状态落地为系统一等概念

长会话若主要靠模型自己“记得住”，基本不必再看。

B.3 工具与审批清单 (invariants)

assert tool = schema 化接口, additional_properties=false
 assert approval policy 可独立评估 (不混在代码 if/else 里)
 assert 危险工具 (Bash 等) 有专门治理 # 非一视同仁
 assert {workdir, network, sandbox, approval} 可显式表达

系统只能回答”我们也有权限控制”，基本等于没设计权限。

B.4 本地治理清单 (invariants)

assert 本地规则可按 {目录, 团队, 任务类型} 分层
 assert skill 视为可复用制度切片, 非长 prompt # 有版本/来源
 assert hook 挂在明确生命周期事件 (pre/post/session_start/stop)
 assert {skill, rule, hook} 具备 {version, source, trigger boundary}

B.5 多代理与验证清单 (invariants)

assert 多代理的第一目的是职责分离, 并行是附带 # 否则是把混乱并行化
 assert 存在独立 verifier (verifier ≠ implementer)
 assert 委派为显式工具或显式状态事件, 非 runtime 魔法
 assert 子代理 {failure, timeout, cancel} ⇒ 明确 cleanup owner

B.6 你更像哪一类系统

更像 Claude Code 的信号:

- 你最重视 query loop、工具编排、中断、compact 与恢复
- 你擅长让规则快速进入现场会话
- 你更关心 agent 如何在复杂任务里持续运行

更像 Codex 的信号:

- 你最重视 instruction fragment、tool schema、approval policy、thread / roll-out / state

- 你擅长把本地规则做成结构化资产
- 你更关心 agent 如何在组织里被长期治理

更像半成品的信号：

- 两边的名词都说得出来，但谁负责秩序说不清
- 有很多能力入口，但没有清楚的恢复路径
- 有很多规则文本，但没有作用域和优先级
- 有多代理，但没有责任分离和收尾机制

B.7 最后六问

要是时间不够，只问这六句：

- 谁拥有最终控制权，模型还是 harness
- 连续性主要住在 loop 里，还是住在线程和状态里
- 工具动手前，谁来拦最后一道
- 本地规则怎么进入系统，怎么分层
- 验证由谁负责，如何独立
- 出事以后，团队靠什么追溯

这六句问下来，系统大概属于哪一派，通常也就露出来了。

B.8 阈值与顺序速查 (thresholds & orderings)

名称	值	作用	源引用
MAX_ENTRYPOINT_LINES	1000	入口文件行数上限	书一 ch5 / memdir/memdir.ts
MAX_SECTION_LENGTH	2000	session memory 单节上限	SessionMemory/prompts.ts
MAX_TOTAL_SESSION_MEMORY_TOKENS	1000000	会话记忆总预算	SessionMemory/prompts.ts
AUTOCOMPACT_BUFFER_TOKENS	30000	autocompact 警戒缓冲	compact/autoCompact.ts
MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES	3	容断阈值	compact/autoCompact.ts

名称	值	作用	源引用
yield_time_ms	per-tool	单次 exec 阻塞毫秒 上限	local_tool.rs
wait_agent.timeout	min/default/max	子代理等待窗口	agent_tool.rs
Bash 子命令数 cap	implicit	复合子命令上限	bashPermissions.ts

事件顺序速查：

- session_start → user_prompt_submit → pre_tool_use → tool exec → post_tool_use → stop
- spawn_agent → send_input* → wait_agent → close_agent (cascade 关闭后代)
- PTL → collapse → reactive compact → 若仍 PTL 则 surface 错误（不再循环）